DAG Mempool: Narwhal-Style Leaderless Transaction Dissemination

Zach Kelling

Lux Industries

2026-06-03

Abstract

The traditional single-mempool-per-validator model caps aggregate transaction throughput at min(validators). We present a DAG mempool where every validator continuously emits ZAP headers carrying transaction hashes; bodies are stored in the Kademlia DHT; aggregate throughput becomes $\sum$(validators). Total ordering of the DAG is deferred to LP-209 (Mysticeti) so that the construction surface is decoupled from the consensus that orders it. This paper expands on LP-208 (`lps/LP-208-dag-mempool.md`) with the wire format for the two new ZAP schemas (`0xF0` DAGHeader, `0xF1` DAGBody), the parent-linking discipline, the throughput model parameterized by per-validator emission rate and validator-set size, the garbage collection contract, and the composition with LP-201 (ZAP P2P), LP-205 (pipelined Quasar), LP-209 (Mysticeti ordering), and LP-217 (cert profile modes).

Contents

1 Introduction

This paper expands on LP-208 (`lps/LP-208-dag-mempool.md`) with detail on the wire format for the two new ZAP schemas (`0xF0` DAGHeader, `0xF1` DAGBody), the parent-linking discipline, the throughput model, the garbage collection contract, and the composition with sibling LPs.

1.1 What was wrong before

Today's Lux mempool model: each validator maintains its own ranked heap of pending transactions; a leader at each round picks from its local mempool when proposing a block. Aggregate throughput is bounded by the slowest validator – if one validator's mempool can admit 50 000 tx/s, that is the aggregate ceiling no matter how many validators participate.

Three concrete failures:

1. **Throughput is min-bounded.** Aggregate admission rate at a chain is min(per-validator-admission). For a 64-validator chain with one validator on consumer hardware (10 000 tx/s) and 63 on Blackwell (200 000 tx/s), aggregate is 10 000 tx/s – 99.2% of the validator-set capacity is wasted.
2. **Tx loss on leader byzantine.** A leader at round R picks from its local mempool. If the leader is byzantine and withholds its block, all txs in its local mempool are stuck until the next round; honest validators' versions of those txs may have already expired.
3. **Latency is leader-rotation-dependent.** A tx admitted to validator V's mempool at time T does not propagate into a block until V is the leader at some future round. The gap is bounded by leader-rotation period; for round-robin with N validators and block period P, the worst-case tx-to-block latency is $N \times P$.

1.2 What this LP solves

The DAG mempool inverts the model. Every validator continuously broadcasts a stream of ZAP headers; each header carries a list of tx hashes, parent pointers to the most recent headers from peer validators, and a leader-signed digest. The actual tx bytes live in the Kademlia DHT (LP-201 layer C), content-addressed by hash. The aggregate structure is a directed acyclic graph: every header has parents from peers, so the graph captures the partial order of tx admission across the validator set.

Throughput becomes $\sum(\text{validators})$. If each validator emits $50\,000\,\text{tx} - \text{hashes/s}$, a 64-validator set admits $3.2\,\text{Mtx} - \text{hashes/s}$ into the DAG. The bottleneck moves from mempool admission to DHT throughput and signature-verification rate per validator – both linearly scalable.

1.3 Scope of this LP

DAG total-ordering – the step that picks a single canonical sequence from the DAG for execution – is deferred to LP-209 (Mysticeti, [1]). This LP defines only the construction surface: the wire format for headers and bodies, the parent-linking discipline, the DHT-storage contract for bodies, the garbage-collection rule for old headers, and the per-validator emission rate.

This separation is deliberate. Multiple ordering algorithms can consume the same DAG: Mysticeti (committee-based ordering with sub-second finality); Bullshark [2] (round-robin anchored ordering with FIFO fairness); Tusk (asynchronous ordering with worst-case finality). Each algorithm has a different latency/throughput/fairness tradeoff. The DAG construction is invariant across these choices.

2 The DAG Structure

A continuously-growing directed acyclic graph. Vertices are headers; edges are parent-pointers. The structure is replicated at every validator and converges via the DHT.

2.1 Vertex (header) properties

- **Unique identifier:** `sha256(header.Bytes())`
- **Validator-local:** each header is signed by exactly one validator
- **Self-stamped:** contains the validator's local wall-clock at emission
- **Parent-linked:** contains one parent-pointer per other validator (best-effort – see parent-linking discipline below)
- **Tx-bearing:** carries a list of tx hashes for the txs the validator admitted since its last header

2.2 Edge properties

- **Cross-validator:** a header from validator V has parent-pointers to recent headers from validators $\neq V$
- **Antiquated-tolerant:** a parent-pointer to a header that has aged out of the active DAG window is treated as a no-op; the DAG remains acyclic
- **Quorum-target:** each header attempts to point to at least $\lceil 2N/3 \rceil + 1$ parents (validator-quorum), but liveness is preserved if fewer parents are reachable

2.3 Emission rate

Each validator emits a new header every K milliseconds.

- **Default K:** 50 ms (configurable per chain via `dag.emission_ms`)
- **Lower bound K:** bounded by network RTT among validators; for an intra-region validator set, K can be as low as 10 ms; for a globally-distributed set, K is typically 100 ms–200 ms
- **Upper bound K:** bounded by the cert timeout in the consumer consensus (LP-217 mode + LP-205 pipeline); K must be lower than the cert wall-clock to avoid serializing on header emission

2.4 Parent-linking discipline

Each header from validator V emitted at time T contains parent pointers chosen as follows:

1. For each other validator W in the active validator set, V maintains a "freshest known header from W" pointer.
2. At header-emit time T, V snapshots its freshest-known set and includes the hashes as `parent_hashes`.
3. If V has no fresh header from W (e.g., W is partitioned, or T is within the first K ms of the epoch), the corresponding slot is zero-padded.

The discipline is *honest-best-effort*. A byzantine V can omit parents, lie about freshness, or zero-pad maliciously; the LP-209 total-ordering algorithm handles these adversarial cases. This LP only specifies the honest validator's behavior; adversarial robustness is a property of the consumer (LP-209) plus the cert profile (LP-217 mode).

3 Wire Format

Two new ZAP schema IDs allocated for this LP, in the range `0xF0`–`0xFF`. The range is chosen to avoid collision with LP-201's `0xD0`–`0xDF` reservation and with any future rollup-substrate

allocation in the `0xE0`–`0xEF` range that LP-204 leaves open (note: LP-218 takes `0xE0`–`0xEF` for rollup envelopes; LP-208 takes `0xF0`–`0xFF`; no overlap).

3.1 Schema `0xF0`: DAGHeader

Carried on unidirectional QUIC streams (LP-201) from emitter to peers; replicated to the DHT (LP-201 layer C) under `sha256(header.Bytes())`.

Field	Type	Notes
`version`	`uint8`	DAG protocol version; activation initializes to `1`
`validator`	`[20]byte`	NodeID of the emitter
`epoch`	`uint64`	DAG epoch; advances on validator-set change
`seq`	`uint64`	Per-validator monotonic sequence number; gaps allowed
`timestamp`	`int64`	Emitter wall-clock at emission (Unix nanoseconds)
`parent_hashes`	`[]bytes32`	Parent header hashes, one per other validator; zero-padded if no parent available; length-prefixed list
`tx_count`	`uint16`	Number of tx hashes in this header
`tx_hashes`	`[tx_count][32]byte`	List of tx hashes for txs admitted since previous header
`body_hash`	`[32]byte`	`sha256(tx_hashes_bytes)`; separate from header hash to allow body GC without invalidating header references
`sig`	`[96]byte`	BLS sig over the canonical pre-image; matches LP-022 BLS leg curve

Table 1: DAGHeader (schema `0xF0`) wire layout.

The BLS pre-image for the signature is `version` ‖ `validator` ‖ `epoch` ‖ `seq` ‖ `timestamp` ‖ `parent_hashes` ‖ `body_hash`. The `body_hash` is the content-address of the DAGBody keyed in the DHT.

3.2 Schema `0xF1`: DAGBody

Carried on bidirectional QUIC streams (LP-201) on `DHTFindValue` response; stored in the DHT under `body_hash`.

Field	Type	Notes
`version`	`uint8`	DAG protocol version; matches the parent DAGHeader
`tx_count`	`uint16`	Number of txs
`tx_bytes`	`[tx_count][]byte`	Length-prefixed list of full tx bytes – each tx is itself a ZAP frame

Table 2: DAGBody (schema `0xF1`) wire layout.

A DAGBody is content-equivalent to a list of tx bytes. There is no header re-embedded in the body – the body is a pure container so that the same body bytes can serve multiple equivalent DAGHeaders that happen to commit to the same tx-set.

Schema ID	Type	Notes
0xC0–0xCB	chains-VM wire (12 VMs)	LP-186
0xD0–0xDF	ZAP P2P transport	LP-201
0xE0–0xEF	rollup-VM envelopes	LP-218
0xF0	DAGHeader	This LP
0xF1	DAGBody	This LP
0xF2–0xFF	Reserved	DAG extensions (Mysticeti ordering hints in LP-209, Block-STM speculatio

Table 3: Cross-LP schema ID allocation map.

3.3 Schema ID allocation

Schema ID rationale: the original LP-208 draft suggested 0xE0–0xE1 but those collide with the rollup substrate reservation that LP-218 finalizes. This LP uses 0xF0–0xFF instead. The change is mandatory – schema IDs cannot overlap across LPs.

4 Why DAG Mempool Beats Traditional Mempool

Four operational properties, each backed by a specific mechanism.

4.1 Throughput is sum-bounded

A traditional mempool has admission rate min(validator_admit_rate); aggregate is the slowest validator.

The DAG mempool has emission rate $\sum$(validator_emit_rate). Each validator emits its own headers; the DAG-wide rate is the sum. The slowest validator contributes its share; it does not gate the fastest.

Reference numbers (LP-203 GPU verify + LP-202 pipeline depth):

- Per-validator emission: 1 header / 50 ms $\times$1000 tx-hashes/header = 20 000 tx − hashes/s
- 64-validator chain: 1.28 Mtx − hashes/s aggregate
- 256-validator chain: 5.12 Mtx − hashes/s aggregate

Headers contain only hashes; bodies are fetched separately via DHT, so per-validator bandwidth is tx_count $\times$ 32 bytes per emission period $\approx$ 640 kB/s per validator at the reference rate. Trivial network overhead.

4.2 No tx loss on byzantine leader

In a single-mempool model, a tx admitted to a byzantine leader's mempool may be withheld from blocks indefinitely; honest validators do not see it until the next leader proposes.

In the DAG mempool, every validator emits its own admitted txs independently. A tx admitted at any honest validator appears in at least one honest validator's header within K milliseconds, and the body is replicated to the DHT under its content-hash. The leader at the consumer layer reads from the DAG, not from its own mempool, so byzantine-leader-withholds-tx is mechanically impossible.

The cert-profile (LP-217 mode) is unaffected: the consumer LP-205/LP-209 reads the DAG and orders it; the cert is computed over the ordered output, not the DAG itself.

4.3 Content-addressed, no re-encoding

Each header references its body via `body_hash`. The body is the tx bytes verbatim. There is no re-encoding step – the bytes that the validator admitted are the bytes that go in the

DHT and that the consumer reads. This is the LP-200 ZAP stack guarantee (immutable buffer, content-addressed, no codec re-marshal) applied at the mempool layer.

A traditional mempool re-encodes txs on insertion (for storage layout) and on emission (for wire format). The DAG mempool does neither; the LP-022 wire format IS the storage format.

4.4 Latency is leader-independent

In a single-mempool model, tx-to-block latency is leader_rotation_period×position_in_rotation. The worst case for a tx that just missed the current leader is one full rotation.

In the DAG mempool, tx-to-DAG latency is bounded by the validator's emission period K (default 50 ms). The tx is in the DAG as soon as the validator that admitted it emits its next header. The consumer's ordering (LP-209) picks the tx for execution at the next ordering pass; for a typical chain with cert wall-clock 5 ms–15 ms, the tx-to-block latency is dominated by K.

5 Garbage Collection and Throughput Targets

The DAG grows continuously. Without GC, validator memory and DHT storage would grow without bound.

5.1 Header GC

A header at height H is eligible for GC when:

- The consumer (LP-209) has total-ordered the DAG past height H, AND
- The cert (LP-217 mode) covering all transactions hashed in the header has finalized.

Eligible headers are removed from the active DAG window. Their content-hashes remain valid forever (anyone who archives a header can still reference it), but live validators no longer hold them in memory. Validator local storage of GC'd headers falls to "cold" – moved to long-term archival storage; no longer indexed for active parent-pointer resolution.

Default window. 2×cert_timeout_ms worth of headers, plus the maximum LP-209 ordering-delay window. Concrete reference: at K=50 ms, LP-217 PQ-strict mode (cert ∼15 ms), LP-209 Mysticeti (ordering delay ∼ 3 rounds) – active window is 5×50 ms = 250 ms of headers = 5 headers per validator = 320 headers in a 64-validator chain. Trivial memory.

5.2 Body GC

Bodies in the DHT (LP-201 layer C) age out per the LP-201 TTL rule: default 24 h with refresh-on-access. Bodies for headers that have been GC'd at all live validators are still served by archival nodes until the TTL expires.

Bodies for headers still in the active DAG window are pinned (LP-201 content-pinning) so they cannot be evicted from the DHT.

5.3 Restart / catch-up

A validator that restarts re-fetches the active DAG window from peers. Each peer serves its own freshest headers via the unidirectional DAGHeader stream; the restarted validator stitches the parent-pointers to reconstruct the DAG. Bodies are re-fetched from the DHT as needed.

Catch-up latency: bounded by active_window_size × network_RTT. For the reference 320-header window over a 100 ms RTT, catch-up is ∼ 32 seconds – fast enough for routine validator restart.

5.4 Throughput targets

The DAG mempool's per-validator emission rate sets the per-chain throughput ceiling at the admission layer. Reference targets:

Per-validator emit rate	64-validator chain	256-validator chain
10 ktx − hashes/s	640 ktx/s	2.56 Mtx/s
50 ktx − hashes/s	3.2 Mtx/s	12.8 Mtx/s
100 ktx − hashes/s	6.4 Mtx/s	25.6 Mtx/s
1 Mtx − hashes/s	64 Mtx/s	256 Mtx/s

Table 4: Per-validator emission rate × validator count = chain-wide admission rate. **Projected.**

The per-validator emit rate is bounded by:

- **GPU verify** (LP-203). BLS leg-sig verify on Blackwell at ∼500 µs aggregate per N=64; ∼ 2000 sig verifies/sec at full precision. For tx sig verify (per-tx, not per-header), GPU bench is ∼1 Mverifies/s on Blackwell – the per-validator emit rate at this hardware class is ∼1 Mtx − hashes/s.
- **DHT body write throughput** (LP-201). Bounded by replication factor k=20 and network bandwidth. At ∼1 kB per tx and 1 Mtx/s emit, body throughput is ∼1 GiB/s into the DHT per validator – within Blackwell-class NIC bandwidth (200 Gbit/s ≈ 25 GiB/s).
- **Mempool admission** (per-validator local). Bounded by tx-sig verify rate on the local GPU and Block-STM speculator throughput. Both scale with GPU class.

Honest reading. The numbers above are projected from the LP-203 measured bench rows extrapolated to the DAG emission model. Measured DAG-mempool throughput will land in the LP-203 addendum once the implementation tag ships.

6 Composition With Sibling LPs

LP	Composition
LP-022	ZAP wire protocol; the framing under schema IDs `0xF0`/`0xF1`
LP-077	Round-digest binding; used by the consumer LP-209 ordering, not by DAG construction
LP-182	Consensus wire / QuasarCert; the cert that finalizes the LP-209 ordering of this DAG
LP-200	ZAP Stack umbrella; the immutable-buffer guarantee this LP relies on for content-addressing
LP-201	ZAP P2P; schema IDs `0xF0`–`0xFF` do not collide with LP-201's `0xD0`–`0xDF` reservation; Kademlia DHT used for body storage
LP-202	Pipelining contract; per-stage unwind primitives apply to the Select stage that consumes from this DAG
LP-203	GPU-native verify; saturates the per-header sig-verify path
LP-205	Pipelined Quasar (sibling); consumes from this DAG via the Select stage
LP-209	Mysticeti ordering (sibling); the consumer that totally orders this DAG
LP-217	Cert profile modes; the consumer's ordering is committed at the chosen mode

Table 5: Composition of LP-208 with sibling LPs.

6.1 With LP-205 (pipelined Quasar)

The two LPs compose orthogonally:

- LP-205 specifies the per-leader pipeline at depth 3.
- LP-208 specifies the per-validator emission of headers into a DAG.

A pipelined-Quasar chain using a DAG mempool runs:

1. Every validator emits DAGHeader every K=50 ms (this LP).
2. LP-209 Mysticeti totally orders the DAG; the ordered output is the stream of txs.
3. The current leader's Select stage (LP-205) consumes from the Mysticeti output instead of a local mempool.
4. Build (LP-205) constructs a block frame referencing the consumed tx-set.
5. Sign (LP-205) produces a QuasarCert at the LP-217 mode.

The leader's Select stage is the only LP-205 stage that changes; the other two stages are unchanged. The aggregate throughput becomes $\sum$(validator_emit_rate) rather than min(validator_mempool_ra

6.2 With LP-209 (Mysticeti ordering)

The contract LP-208 makes to LP-209:

1. The DAG is content-addressed; LP-209 can refer to vertices by hash and rely on byte-identity across validators.
2. Headers from honest validators carry honest tx-hash lists and honest-best-effort parent pointers; LP-209 specifies how to recover ordering against adversarial headers.
3. Bodies are eventually consistent in the DHT under `body_hash`; LP-209 specifies when to wait for body availability versus proceeding with header-only ordering.

6.3 With LP-203 (GPU verify)

Tx sig verify runs on Blackwell as DAG headers arrive. Each header carries up to 1000 tx hashes; the bodies for those hashes are batched into a single CUDA dispatch on the verify path. Verify throughput tracks the LP-203 bench numbers directly – at ∼1 Mverifies/s on Blackwell, the GPU is not the bottleneck up to a per-validator emit rate of ∼1 Mtx/s.

6.4 With LP-217 (cert profile modes)

The DAG ordering (from LP-209 Mysticeti consuming this LP's DAG) is committed at the LP-217 mode the consumer chain has configured – PQ-fast for typical workloads, PQ-strict for custody/treasury, PQ-heavy for archival anchoring. The DAG itself is mode-agnostic; only the cert that finalizes the ordering of DAG vertices into blocks carries the mode posture.

7 Activation Marker

```
activates:      2025-12-25T16:20:00-08:00
activates-unix: 1766708400
```

DAG mempool is opt-in at activation. A chain enables it via genesis config:

```
dag_mempool:
  enabled: true
  emission_ms: 50
  active_window_headers: 320
  schema_ids:
    header: 0xF0
    body: 0xF1
```

8 Backwards Compatibility

None. Chains that do not enable DAG mempool use the single-mempool model unchanged. LP-205 (pipelined Quasar) works against either input source.

9 Future Work

- **LP-209** – Mysticeti ordering: total-ordering of the DAG with sub-second finality; this LP's DAG is its input [1].
- **LP-210** – Block-STM speculation: speculative execute against the DAG's tx stream ahead of LP-209 ordering; commit on ordering arrival.
- **LP-211** – Cross-shard atomic commit: DAG mempools across peer shards exchange parent-pointers via cross-shard schema IDs (`0xF2`–`0xFF` reserved for this).

References

[1] Kushal Babel, Andrey Chursin, George Danezis, Anastasios Kichidis, Lefteris Kokoris-Kogias, Arun Koshy, Alberto Sonnino, and Mingwei Tian. Mysticeti: Reaching the limits of latency with uncertified DAGs, 2024. https://arxiv.org/abs/2310.14821.

[2] Alexander Spiegelman, Neil Giridharan, Alberto Sonnino, and Lefteris Kokoris-Kogias. Bullshark: DAG BFT protocols made practical, 2022. CCS 2022; https://arxiv.org/abs/2201.05677.